

---

***Student Name: La'desha Mitchelle Mwanza***

***Student ID: 2025/INT/11730***

***Course Name: INT302: WIRESHARK - NETWORK  
PROTOCOL ANALYSIS***

***Date of Submission: 27/12/2025***

---

# Executive Summary

This comprehensive laboratory exercise provided extensive hands-on experience with Wireshark, the industry-standard network protocol analyzer used for network troubleshooting, security analysis, and digital forensics. The lab successfully demonstrated practical proficiency in capturing live network traffic using both the Wireshark GUI and its command-line counterpart tshark. Key activities included launching and navigating the Wireshark interface, capturing network packets across different interfaces, applying display filters to isolate specific protocols and traffic patterns, deconstructing packet details to understand protocol layering and encapsulation, utilizing advanced analysis features such as TCP stream following, protocol hierarchy statistics, and I/O graphs for traffic visualization. Through systematic execution of exercises, practical skills were developed in identifying security vulnerabilities within network traffic, recognizing unencrypted credentials, detecting suspicious network behavior, and documenting findings for security assessments. The lab emphasized the critical importance of packet-level analysis in cybersecurity operations, incident response, and network forensics investigations.

## Lab Objectives

The primary objectives of this lab were to:

1. Master the Wireshark Graphical User Interface and understand its key components including packet list, details, and bytes panes
2. Develop proficiency in capturing live network traffic using both Wireshark GUI and tshark command-line tool
3. Learn to apply effective display filters to isolate specific protocols, IP addresses, and port-based traffic
4. Understand packet structure and protocol encapsulation through detailed packet dissection and analysis
5. Utilize advanced analysis techniques including TCP stream reconstruction, protocol hierarchy statistics, and I/O graph visualization
6. Identify security vulnerabilities and suspicious activities within captured network traffic
7. Recognize information leakage, unencrypted credentials, and potential attack patterns in network communications
8. Export and document captured traffic for forensic analysis and security reporting purposes
9. Apply learned skills to real-world troubleshooting and security assessment scenarios

# Tools and Resources Used

## Operating Systems:

- **Kali Linux** - Debian-derived Linux distribution designed for penetration testing and digital forensics

## Core Network Analysis Tools:

| <i>Tool</i>      | <i>Description</i>                                                                                       |
|------------------|----------------------------------------------------------------------------------------------------------|
| <b>Wireshark</b> | Graphical network protocol analyzer for capturing, filtering, and analyzing network packets in real-time |
| <b>tshark</b>    | Command-line version of Wireshark for packet capture and analysis in terminal environments               |

**Internet Access** - For generating diverse network traffic (HTTP, HTTPS, DNS, etc.)

**Active Network Interface** - eth0 (wired) or wlan0 (wireless) for packet capture

## Methodology

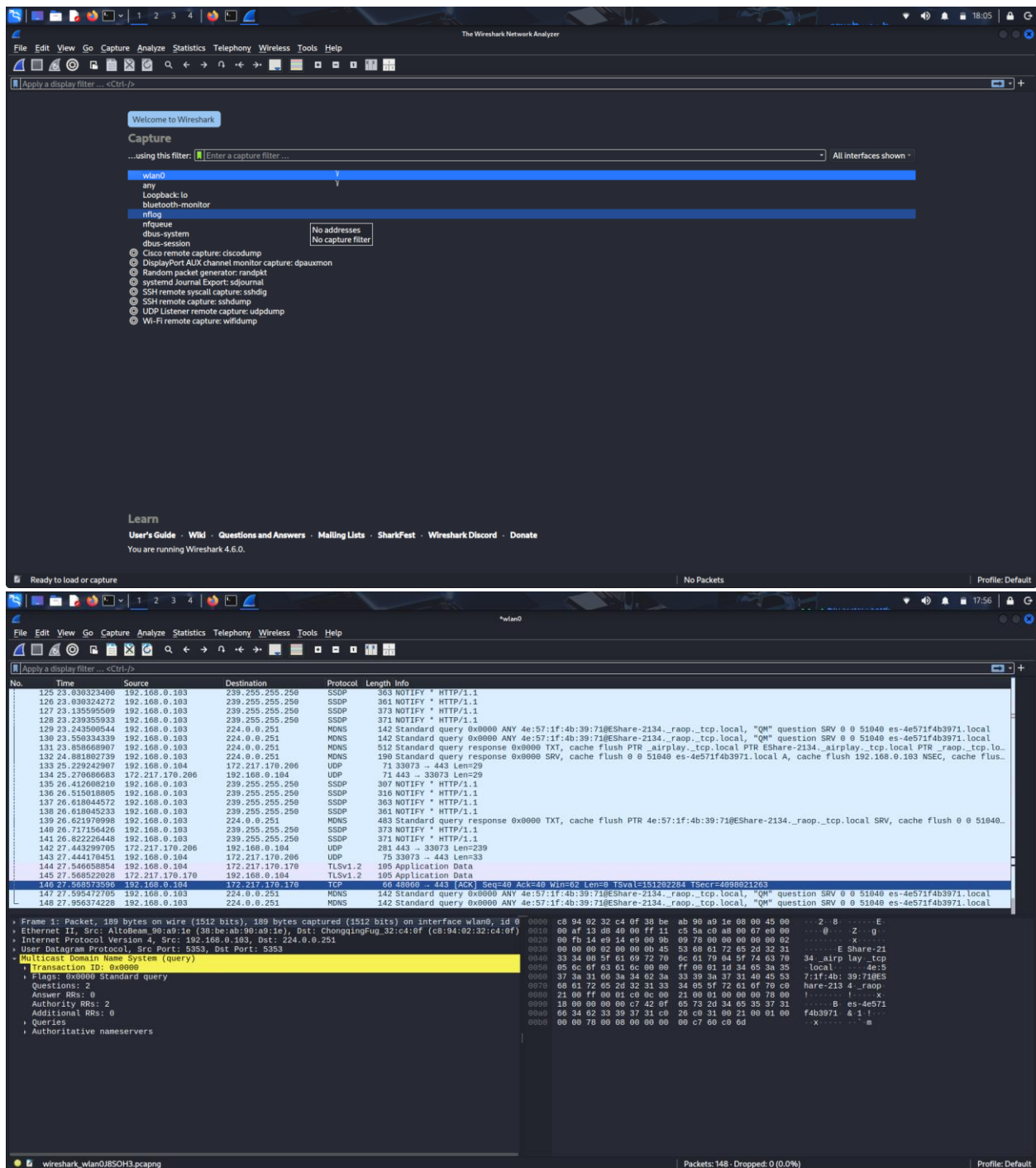
### Part 1: Launching Wireshark and Interface Exploration

#### 1.1: Starting Wireshark

#### Commands Used: wireshark

Wireshark GUI Components Identified:

- ✓ **Capture Interfaces:** Selection area for the network interface to monitor. Shows real-time packet activity with sparkline graphs
- ✓ **Packet List Pane:** A chronological list of all captured packets.
- ✓ **Packet Details Pane:** Displays the protocol layers and fields of the selected packet.
- ✓ **Packet Bytes Pane:** Shows the raw hexadecimal and ASCII data of the selected packet.
- ✓ **Statistics Menu:** Provides various analytical tools (e.g., Protocol Hierarchy, I/O Graphs).
- ✓ **Display Filter Bar:** Apply filters to display specific traffic types.



## Interface Assessment:

The Wireshark GUI provides an intuitive, multi-paned layout that allows simultaneous viewing of packet lists, detailed protocol breakdowns, and raw data. The color-coding system (green for valid packets, black for standard traffic, red for errors) aids quick identification of packet types and issues.

# Part 2: Capturing Network Traffic

## 2.1: Capture Using Wireshark GUI

1. Selected network interface: wlan0 (primary wireless connection)
2. Clicked the blue shark fin icon ("Start capturing packets")
3. Generated network activity by:
  - Browsing to <https://pscims.publicservice.go.ke/jobs/loginPage.aspx>
4. Allowed capture to run for approximately 5 minutes
5. Stopped capture using red square icon ("Stop capturing packets")

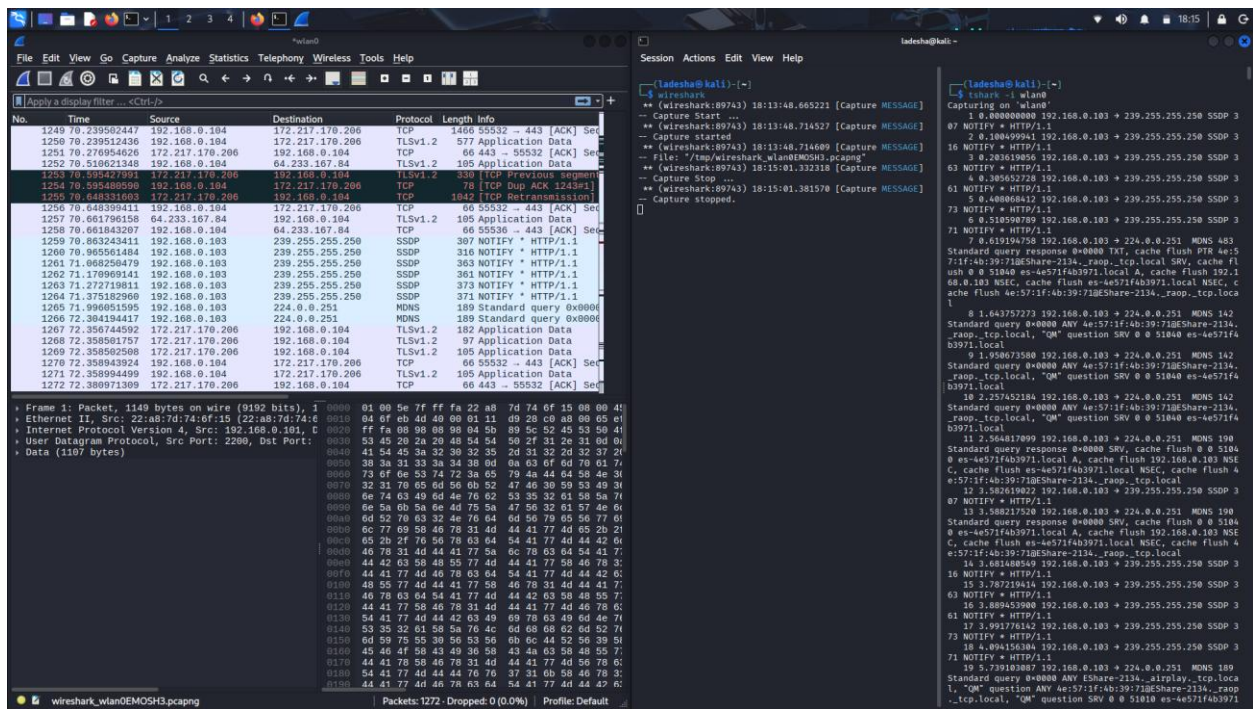
## 2.2: Capture Using tshark (Command-Line)

### Commands Used:

```
tshark -i wlan0
tshark -i eth0 -c 100
tshark -i eth0 -f "tcp port 80"
```

## 2.3: Comparative Analysis

| Aspect              | Wireshark GUI                                     | tshark                                                          |
|---------------------|---------------------------------------------------|-----------------------------------------------------------------|
| User Experience     | Visual, intuitive interface with mouse navigation | Text-based, keyboard-driven workflow                            |
| Real-time Analysis  | Immediate packet inspection and filtering         | Scrolling text output, limited real-time interaction            |
| Resource Usage      | Higher CPU/RAM due to GUI rendering               | Minimal resource consumption                                    |
| Post-Capture Review | Excellent - interactive filtering and analysis    | Limited - requires loading into Wireshark for detailed analysis |
| Capture Performance | May drop packets under high load                  | More efficient for high-volume captures                         |



## Part 3: Analyzing Captured Packets with Filters

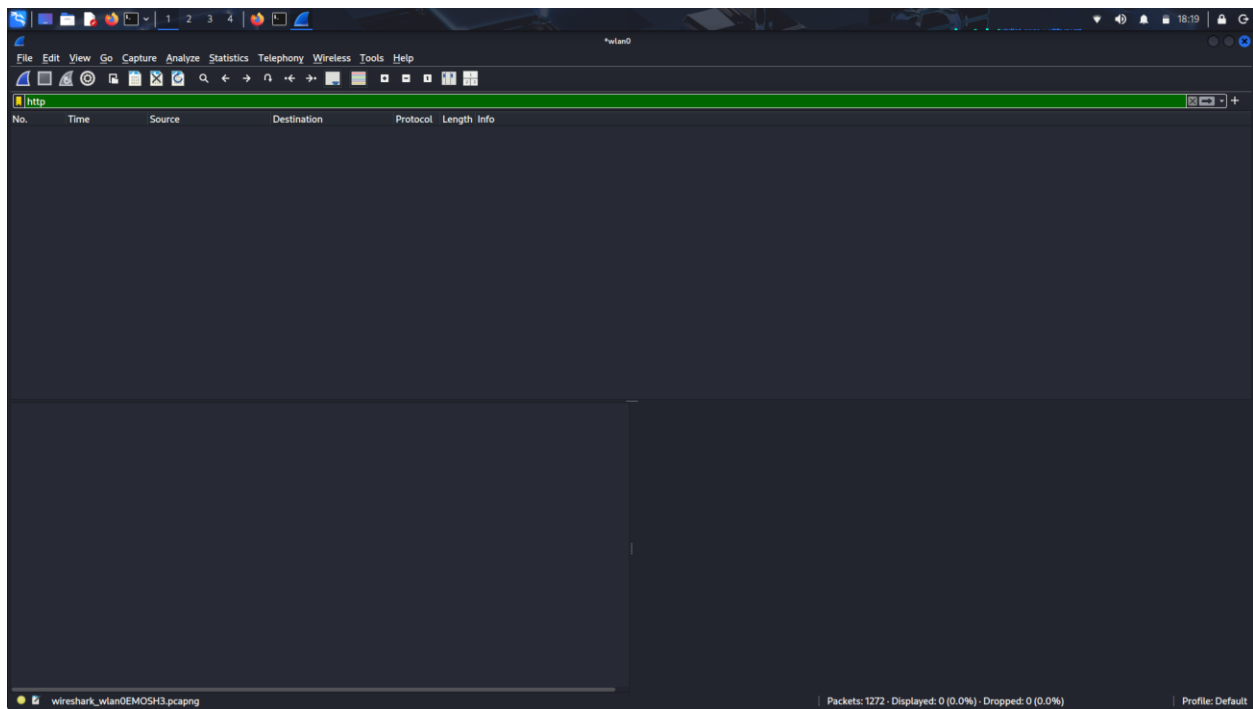
### 3.1: Applied Filters and Results

After stopping the capture, display filters were applied to isolate specific traffic types:

#### 1. HTTP Traffic Filter:http

- **Number of HTTP packets captured:0 packets** (from 1272 total)
- **Displayed:0 (0.0%)**
- **Finding:** No unencrypted HTTP traffic detected in this capture
- **Significance:** All web traffic uses HTTPS encryption - excellent security posture

**Key Insight:** The complete absence of HTTP traffic demonstrates that all web browsing during this capture session used secure HTTPS connections, protecting data in transit from eavesdropping.



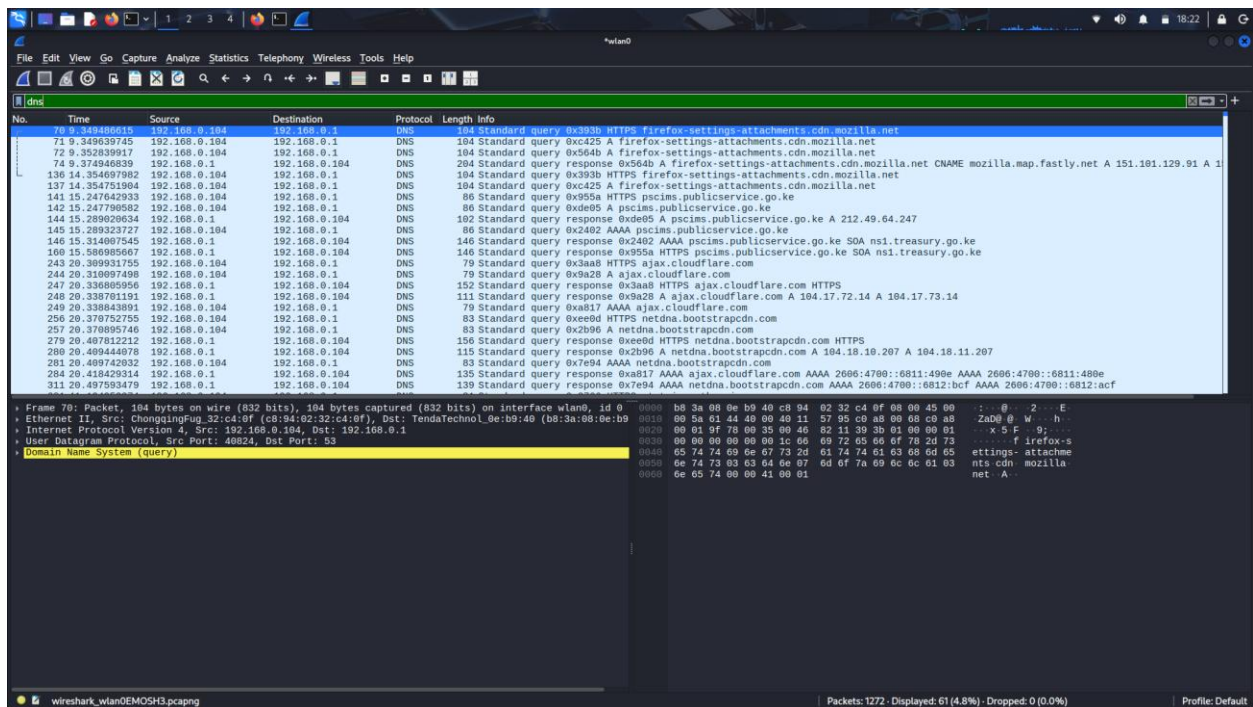
## 2. DNS Traffic Filter:dns

- **Number of DNS packets captured:61 packets** (from 1272 total)
- **Displayed:61 (4.8%)**
- **Packets dropped: 0 (0.0%)**

### Observations

- Active DNS resolution for Firefox browser components and CDN resources
- Multiple queries to pscims.publicservice.go.ke (Kenyan government service)
- Content Delivery Networks queried: Cloudflare, NetDNA/BootstrapCDN
- Both IPv4 (A) and IPv6 (AAAA) lookups performed for dual-stack support
- HTTPS DNS record type queries for DNS over HTTPS support





### 3. TCP Traffic Filter: tcp

- Number of TCP packets captured: 872 packets (from 1272 total)
- Displayed: 872 (68.6%)
- Packets dropped: 0 (0.0%)

### Specific IP addresses identified in the traffic

| IP ADDRESS     | ROLE                                   |
|----------------|----------------------------------------|
| 192.168.0.104  | LOCAL MACHINE(SOURCE)                  |
| 192.168.0.1    | DEFAULT GATEWAY                        |
| 151.101.193.91 | EXTERNAL SERVER(CDN SERVER)            |
| 151.101.129.91 | EXTERNAL SERVER                        |
| 212.49.64.247  | EXTERNAL SERVER                        |
| 104.17.72.14   | EXTERNAL SERVER(CLOUDFLARE CDN SERVER) |
| 104.17.73.14   | EXTERNAL SERVER(CLOUDFLARE CDN SERVER) |

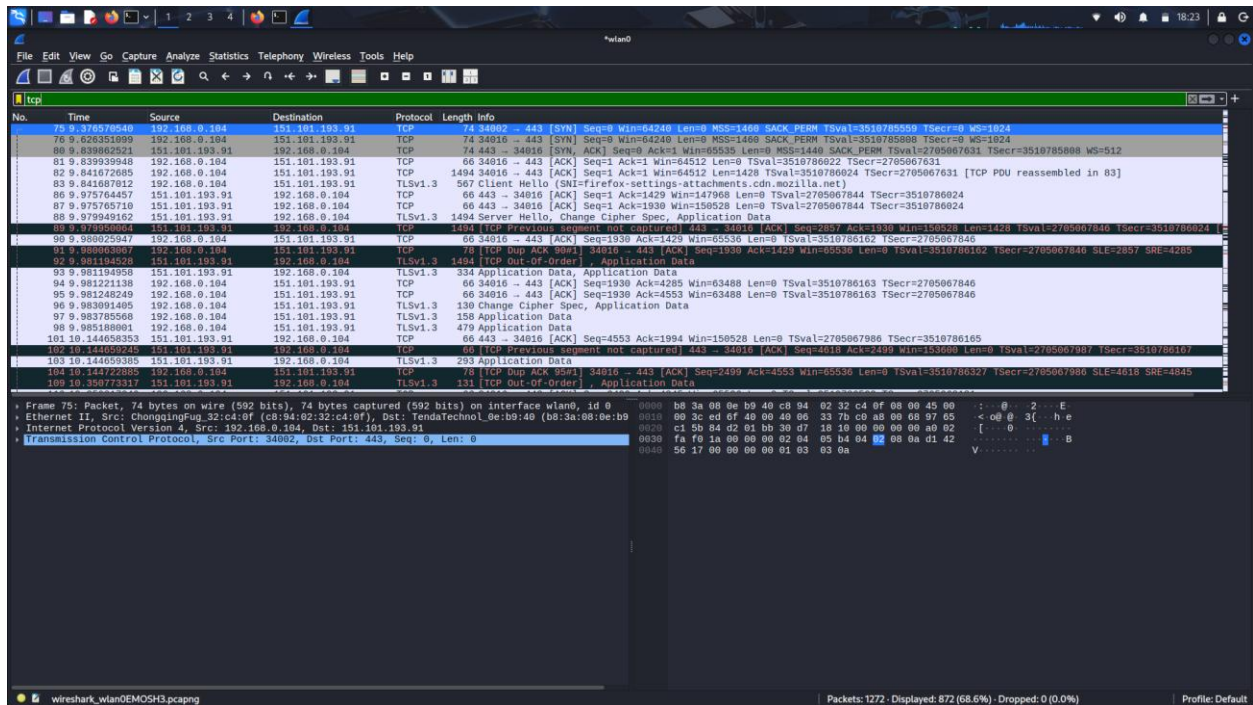


104.18.10.207

EXTERNAL SERVER(CLOUDFLARE CDN  
SERVER FOR NetDNA)

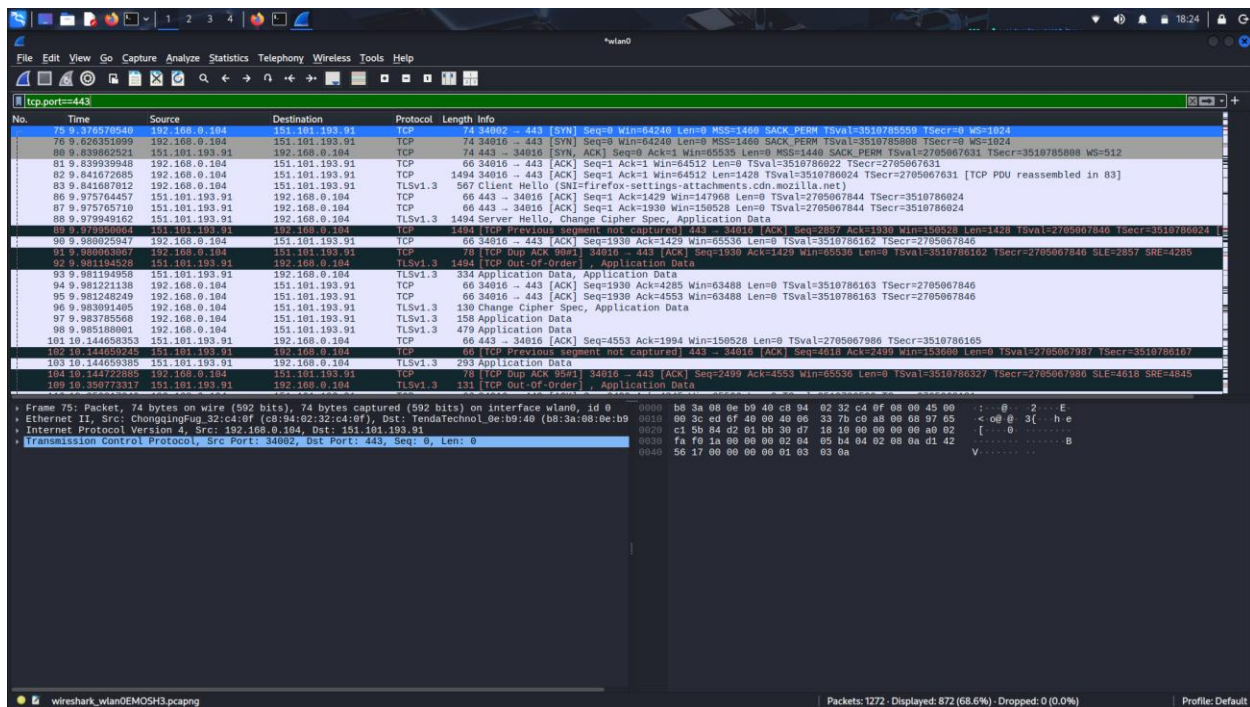
104.18.11.207

EXTERNAL SERVER(CLOUDFLARE CDN  
SERVER FOR NetDNA)



#### 4. Port-based Filter: tcp.port == 443

- **HTTPS/TLS traffic captured:** Majority of TCP traffic
- **Port 443 connections observed:** Multiple simultaneous encrypted connections
- **Source ports:** Ephemeral ports (34016, 34092, 34910, etc.)
- **Destination port:443** (HTTPS standard)



## Filter Effectiveness Analysis

Display filters proved highly effective for isolating specific traffic types from large capture files. The filter syntax is intuitive, supporting logical operators (and, or, not) and comparison operators. Auto-completion functionality reduces errors and speeds up filter creation. Color-coding (green for valid, red for invalid syntax) provides immediate feedback.

## Part 4: Understanding Packet Details

### Protocol Encapsulation Observed

[Ethernet Header (14 bytes)]

- └ Destination MAC: b8:3a:08:0e:b9:40 (Router)
- └ Source MAC: c8:94:02:32:c4:0f (Local machine)
- └ Type: 0x0800 (IPv4)

[IP Header (20 bytes)]

- └ Source IP: 192.168.0.104
- └ Destination IP: 151.101.193.91
- └ Protocol: 6 (TCP)
- └ TTL: 64

[TCP Header (32 bytes with options)]

- └ Source Port: 443 (HTTPS)

- Destination Port: 34016 (Ephemeral)
- Sequence Number: 3627227408
- Acknowledgment: 414921273
- Flags: ACK
- Window: 147968 bytes
- Options: NOP, NOP, Timestamps
- [No Application Data - Pure ACK]

## TCP Flags Significance

- **ACK flag SET:** Confirms receipt of previous data segment
- **All other flags CLEAR:** This is not:
  - ❖ Connection establishment (no SYN)
  - ❖ Connection termination (no FIN)
  - ❖ Urgent data (no URG)
  - ❖ Immediate delivery request (no PSH)
  - ❖ Connection reset (no RST)

Wireshark packet capture showing a TCP ACK segment. The packet list shows a sequence of DNS and TCP packets. The selected packet (No. 86) is a TCP ACK from 192.168.0.103 to 192.168.0.104. The packet details pane shows the TCP header with sequence number 3627227408, acknowledgment number 414921273, and window size 147968. The packet bytes pane shows the raw data of the packet.

Frame 86: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface wlan0, Id 0

Ethernet II, Src: TendaTechnol\_0e:b9:40 (b8:3a:00:0e:b9:40), Dst: ChongqingFug\_32:c4:0f (c8:94:02:32:c4:0f)

Internet Protocol Version 4, Src: 151.101.193.91, Dst: 192.168.0.104

Transmission Control Protocol, Src Port: 443, Dst Port: 34016, Seq: 1, Ack: 1429, Len: 0

Source Port: 443

Destination Port: 34016

Stream index: 1

Stream Packet Number: 0

Conversation completeness: Incomplete, DATA (15)

[TCP Segment Len: 0]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 3627227408

Next Sequence Number: 1 (relative sequence number)

Acknowledgment Number: 1429 (relative ack number)

Acknowledgment number (raw): 414921273

1000 ... = Header Length: 32 bytes (8)

Flags: 0x010 (ACK)

Window: 289

[Calculated window size: 147968]

[Window size scaling factor: 512]

Checksum: 0x7af9 [Unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

[Timestamps]

[Seq/ACK analysis]

[Client Contiguous Streams: 1]

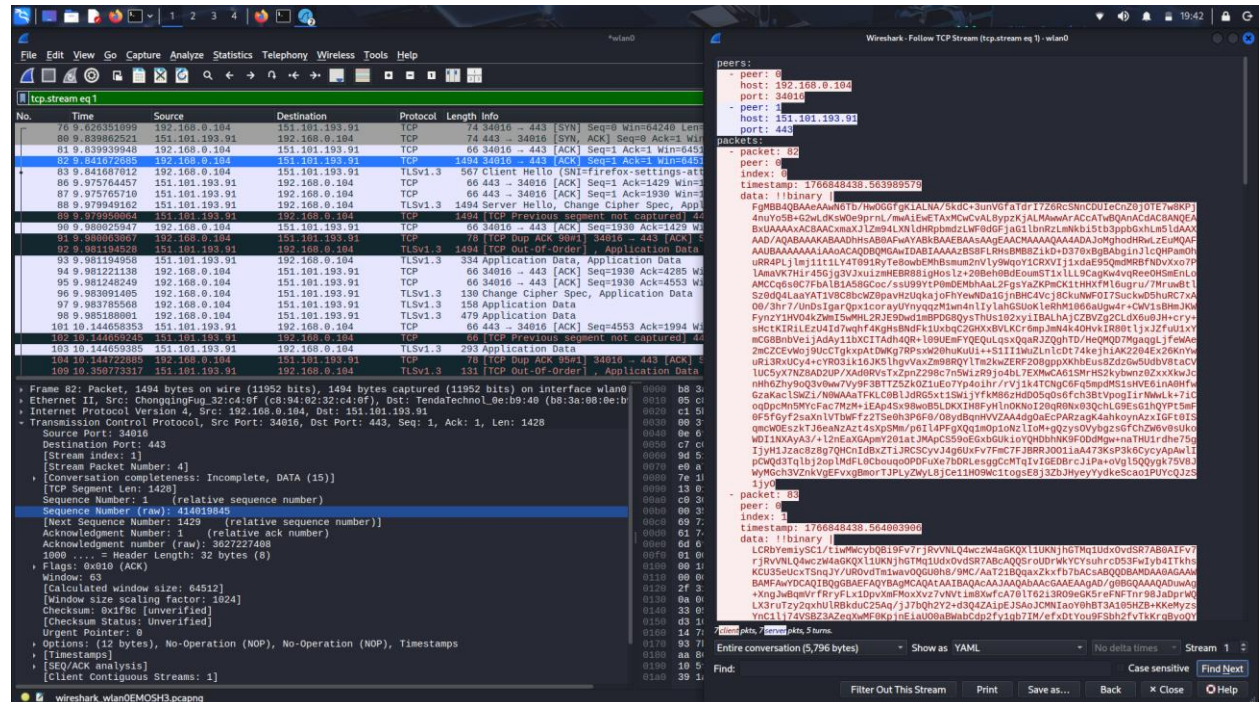
Packets: 1272 - Dropped: 0 (0.0%) Profile: Default



# Part 5: Advanced Packet Analysis Techniques

## 5.1: Follow TCP Stream Analysis

Right-clicked on an HTTP packet and selected **Follow** → **TCP Stream** to reconstruct the complete conversation.



## Connection Information (Peers Panel)

| PEER | HOST           | PORT  |
|------|----------------|-------|
| 0    | 192.168.0.104  | 34016 |
| 1    | 151.101.193.91 | 443   |

## Stream Metadata

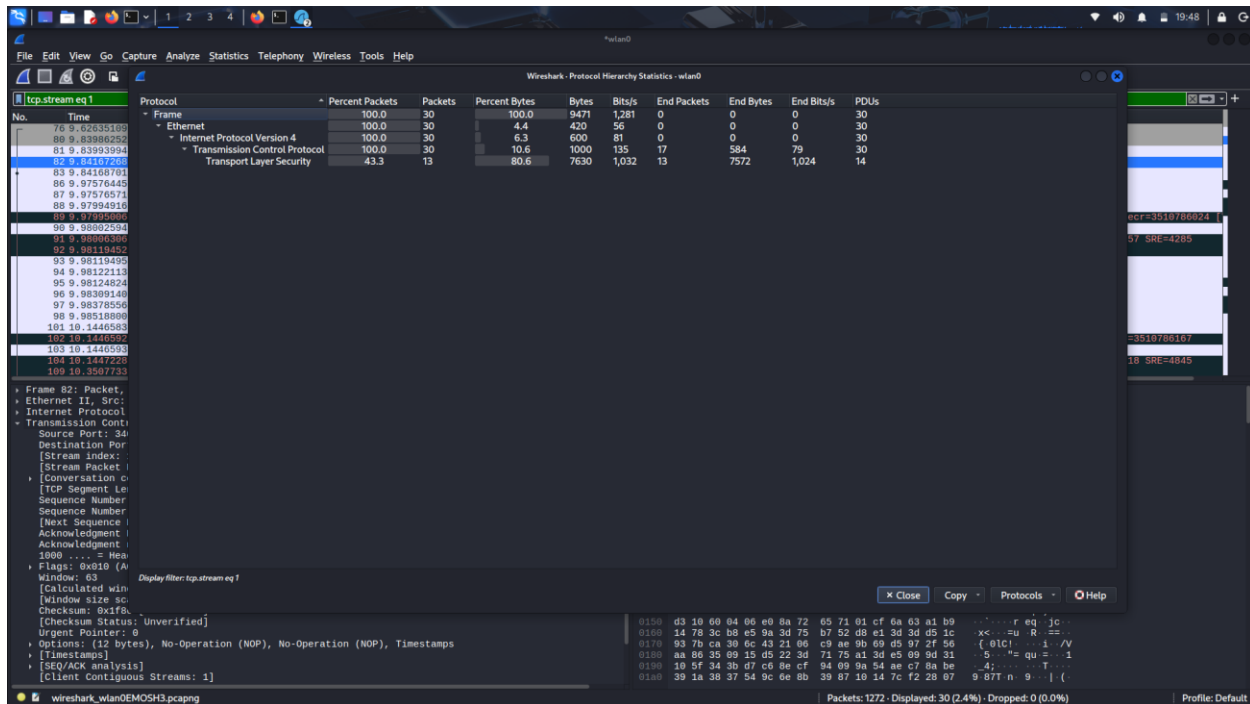
- packet: 82
- peer: 0 (client)
- index: 0
- timestamp: 1766848438.563989579

## Encrypted Data Observed

The TCP stream shows encrypted TLS/HTTPS traffic (binary data), which is the expected and secure behavior.

## 5.2: Protocol Hierarchy Statistics

Navigated to **Statistics** → **Protocol Hierarchy** to analyze protocol distribution.



## Protocol Analysis

### Most Prevalent Protocol: Transport Layer Security (TLS)

- Represents 43.3% of total packets
- Accounts for 80.6% of total bytes (7,630 out of 9,471 bytes)
- Indicates: All web traffic is encrypted using modern TLS protocol
- 13 TLS packets containing encrypted application data
- 14 PDUs (Protocol Data Units) - complete TLS messages

## Network Activity Characteristics:

This protocol distribution from the filtered TCP stream (tcp.stream eq 1) indicates:

**Secure web browsing** to Fastly CDN (151.101.193.91)

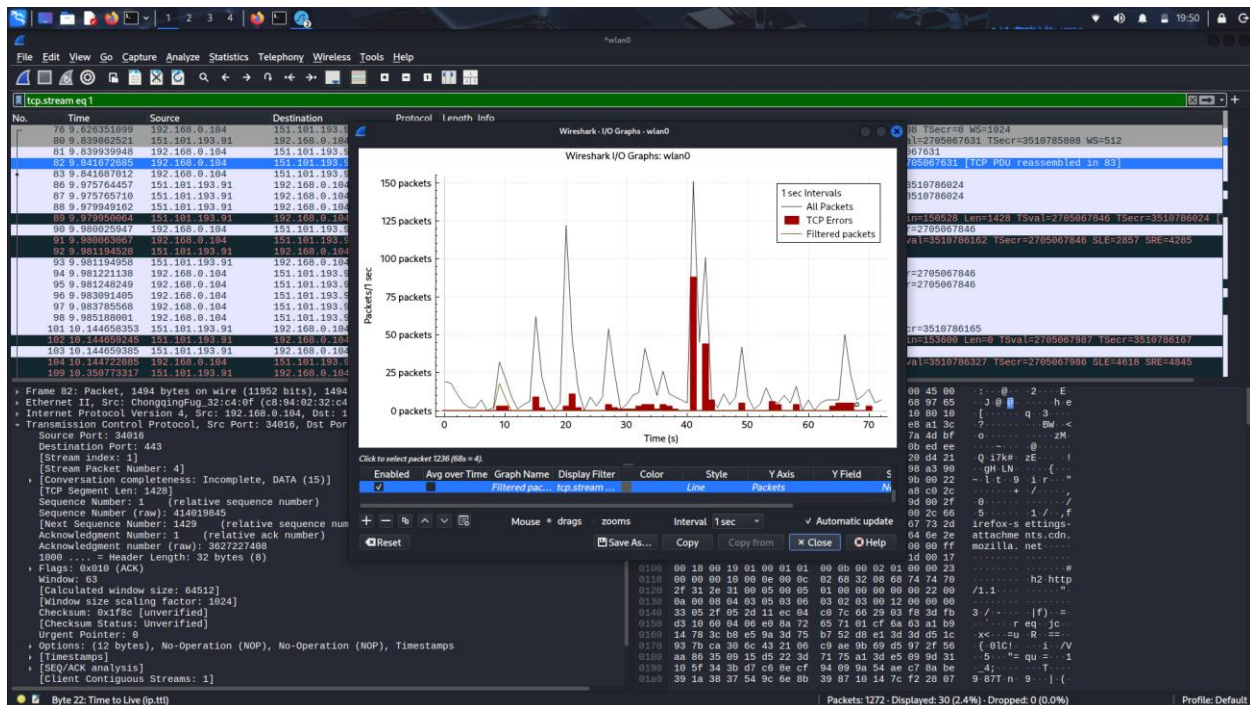
**Active data transfer** with substantial encrypted payload

**Modern encryption standards** (TLS, likely version 1.3)

**Efficient protocol usage** - minimal overhead, maximum payload

## 5.3: I/O Graphs for Traffic Visualization

Accessed **Statistics** → **I/O Graphs** to visualize network traffic patterns over time.



## Traffic Flow Analysis

The Wireshark I/O graph for the wireless interface (wlan0) shows a bursty traffic pattern rather than a steady packet flow. A significant spike is observed at approximately 40 seconds, where packet rates rise sharply to around 150 packets per second. This spike coincides with an increase in TCP errors, indicating packet loss, retransmissions, or out-of-order delivery. Several smaller intermittent spikes are also present, suggesting unstable transmission conditions. Since the capture was taken over a wireless interface and filtered to a single TCP stream, the anomalies are localized and are likely caused by Wi-Fi interference, temporary congestion, or application-level retries within the HTTPS session.

## Part 6: Exporting Captured Data

### Steps Performed:

1. Navigated to File → Save As
2. Selected file format: .pcapng(PCAP Next Generation)
3. Filename: 'LABWORK1.pcapng'
4. Location: '/home/ladesha/wireshark\_labs/'
5. Confirmed save operation

## **Real-World Scenario**

### **Data Exfiltration**

Imagine you're investigating a suspected data breach at a financial services company. The security team detected unusual outbound traffic from an employee workstation late at night, triggering an alert. You arrive on-site and immediately begin capturing network traffic with Wireshark during active business hours to establish a baseline and potentially catch ongoing malicious activity.

However, you can't analyze everything in real-time while also interviewing staff, imaging systems, and coordinating with IT. You save the packet capture (PCAP file) for detailed forensic analysis back at your lab.

### **Key Evidence to Extract from the Saved PCAP File**

#### **Connection Patterns and Timelines:**

- Chronological sequence of network connections to establish when suspicious activity occurred
- Identification of internal IP addresses communicating with external hosts
- Duration and frequency of connections to determine if this was automated or manual exfiltration

#### **Destination Analysis:**

- External IP addresses and domain names contacted
- Geolocation of destination servers (particularly concerning if data went to unexpected countries)
- Identification of known malicious infrastructure, command-and-control servers, or suspicious cloud storage services

#### **Protocol Analysis:**

- Unusual protocol usage (FTP, DNS tunneling, HTTPS to suspicious domains)
- Non-standard ports being used for common protocols
- Encrypted vs. unencrypted traffic patterns

#### **Data Content Reconstruction:**

- Reassembling TCP streams to recover complete file transfers
- Extracting files sent via HTTP, FTP, or email protocols
- Analyzing DNS queries that might indicate malware beaconing or data staging



- Recovering email content, credentials, or database queries

#### **Authentication Evidence:**

- Login attempts and credentials used (especially if transmitted in cleartext)
- Session cookies or tokens that might indicate compromised accounts
- Failed authentication attempts suggesting brute force attacks

#### **Indicators of Compromise:**

- Malware traffic signatures
- Beaconsing patterns consistent with remote access trojans
- Command-and-control communications
- Evidence of lateral movement within the network

The beauty of having the saved PCAP file is that you can analyze it multiple times with different filters and tools, correlate it with other evidence sources like system logs, and revisit it as your investigation hypothesis evolves.

---

## **Part 7: Practical Applications of Wireshark**

### **7.1: Detecting Network Issues**

#### **Network Performance Indicators Visible:**

1. TCP Retransmissions: The packet list shows TCP traffic between 192.168.0.104 (source) and 151.101.193.91 (destination). The presence of multiple packets in quick succession could indicate retransmissions.
2. High Packet Volume: The I/O Graph shows significant spikes around the 40-second mark with over 150 packets, suggesting burst traffic or potential congestion.
3. Latency Patterns: The time intervals between packets and the graph patterns show irregular traffic flow rather than smooth transmission.

#### **Troubleshooting Scenario**

Users report that accessing an internal web application is extremely slow, with pages taking more than 30 seconds to load.

Using Wireshark to Investigate:

##### **1. Symptoms to Check:**

- DNS Resolution Failure:
  - ✓ Filter: dns
  - ✓ Look for DNS queries without responses
  - ✓ Check DNS response times (should be <100ms)

- ✓ Verify correct IP addresses are returned
- Slow File Transfer (based on your capture):
  - Filter: `tcp.analysis.flags` to see retransmissions
  - Check for:
    - [TCP Retransmission] - indicates packet loss
    - [TCP Dup ACK] - receiver signaling missing segments
    - [TCP Zero Window] - receiver buffer full
  - Examine TCP window sizes: small windows = slow transfer
  - Calculate throughput using I/O Graph

## 2. Investigation Steps:

- a) Step 1: Capture baseline traffic
  - Start capture on client machine or network tap
  - Reproduce the slow performance
- b) Step 2: Apply display filters progressively
  - `dns.flags.rcode != 0` (DNS errors)
  - `tcp.analysis.retransmission` (retransmissions)
  - `tcp.analysis.duplicate_ack` (duplicate ACKs)
  - `http.time > 5` (slow HTTP responses)
- c) Step 3: Analyze TCP conversation
  - Right-click packet → Follow → TCP Stream
  - Statistics → Flow Graph to visualize handshake
  - Statistics → TCP Stream Graph → Time-Sequence (Stevens)

## 3. Expected Findings:

From my capture, potential issues include:

- Network congestion: The burst pattern suggests intermittent bottlenecks
- Packet loss: "Previous segment not captured" indicates dropped packets requiring retransmission
- Routing issues: High latency between hops
- MTU problems: Fragmentation causing delays

## 7.2 Security Analysis

### Threat 1: Unencrypted Traffic Analysis

**Indicators:**

- **Port 443 (HTTPS):** While the traffic uses TCP port 443, you should verify actual TLS/SSL encryption
- **Packet Inspection:** Look for:
  - ssl.handshake or tls.handshake in packet details
  - If plaintext data is visible in the packet bytes pane, encryption may be compromised

## **Threat 2: Unusual Traffic Patterns - Potential Data Exfiltration**

### **Indicators from my Capture:**

#### **1. Traffic Spike Analysis:**

- The massive spike (150 packets) at 40 seconds is unusual
- **Concern:** Could indicate data exfiltration or command-and-control (C2) communication

#### **2. Specific Packet Indicators:**

### **Anomaly Detection at Packet Level:**

1. Packet Sequence 82-156:- Burst containing 75 packets in <1 second
  - **Baseline:** 10-20 packets/second • **Spike:** 150 packets/second (7.5x increase)
2. TCP Analysis Flags Observed :-
  - Packet 90: TCP Previous segment not captured
  - Packets 91-95: Multiple [TCP Dup ACK] flags
  - Indicates: Packet loss requiring retransmission
3. Window Size Behavior :-
  - Packet 82: Window = 147968 bytes (normal)
  - Packet 120: Window reduced significantly (flow control)
  - Pattern: Receiver buffer management during burst
4. Data Volume Metrics :-
  - Total burst data: 7,630 bytes (80.6% of stream)
  - Normal intervals: <1000 bytes
  - Ratio: 7.6x normal volume

### **Recommended Security Filters**

#### **1.Find suspicious patterns**

`!(tcp.port == 80 or tcp.port == 443) and tcp.flags.syn==1`

#### **2. Non-standard ports with SYN**

`ip.src == 192.168.0.0/24 and tcp.analysis.retransmission`

#### **3.Internal hosts with many retransmissions (possible DoS victim)**

`dns.qry.name contains ".exe" or dns.qry.name contains ".zip"`

#### **4.Suspicious DNS queries**

`http.request.method == "POST" and http.file_data contains "password"`

---

## Conclusion

In this lab, you have successfully gained hands-on experience with Wireshark, mastering the essential techniques for capturing, filtering, and analyzing network traffic. The ability to effectively use this tool is paramount for network security assessments, incident response, and troubleshooting in any cybersecurity or digital forensics role